

Sistemi Operativi – Modulo Laboratorio

Relazione del Progetto: Booking System

A.A. 2025/26

Feri Alessandro – Matricola 7081870

Introduzione

Il progetto consiste nella realizzazione di un Booking System client-server per la gestione di prenotazioni relative a risorse condivise, come aule, libri, visite o servizi simili.

Il sistema permette a più client di collegarsi al server, autenticarsi, inviare richieste di prenotazione e consultarne lo stato. Gli utenti standard possono registrarsi, effettuare il login, creare nuove richieste e visualizzare solo le proprie prenotazioni. L'amministratore, invece, ha privilegi aggiuntivi: può visualizzare tutte le richieste, modificarne lo stato e controllare eventuali conflitti.

Il server gestisce le connessioni tramite socket TCP/IPv4 e adotta un modello thread-per-client, così da servire più utenti contemporaneamente. Per mantenere la consistenza dei dati condivisi, le operazioni critiche sugli utenti e sulle prenotazioni sono protette tramite mutex.

Le informazioni vengono mantenute in memoria tramite array di dimensione fissa e salvate su file di testo, in modo da non perdere utenti e prenotazioni alla terminazione del programma. La comunicazione tra client e server avviene tramite un semplice protocollo testuale del quale le costanti sono definite in `protocol.h`.

Architettura

Per il progetto ho adottato un'architettura client-server. Il server è il componente centrale: mantiene l'archivio degli utenti e delle prenotazioni, resta in ascolto sulla porta 8080 e gestisce le richieste ricevute dai client.

Il client è invece un programma da terminale che fornisce un menu interattivo. Il suo compito è raccogliere l'input dell'utente, costruire i comandi secondo il protocollo definito e mostrare le risposte ricevute dal server.

In questo modo la logica principale dell'applicazione rimane lato server, mentre il client svolge solo il ruolo di interfaccia verso l'utente.

Protocollo di comunicazione

Ho definito un protocollo custom per lo scambio dei dati tra server e client. Ho basato la comunicazione tra client e server su messaggi testuali. Ogni richiesta segue il formato generale:

```
COMANDO|argomento1|argomento2|...
```

Il primo campo identifica il comando da eseguire, mentre i campi successivi (massimo 4) rappresentano gli eventuali argomenti. I comandi sono definiti in `protocol.h` come costanti, in modo da evitare stringhe duplicate e errori di battitura nel codice.

I comandi principali implementati sono i seguenti:

Comando	Utilizzatore	Argomenti	Descrizione
REGISTER	tutti	username, password	Registra un nuovo utente
LOGIN	tutti	username, password	Autentica un utente o l'amministratore
BOOK	utente	risorsa, data, ora inizio, ora fine	Invia una nuova richiesta di prenotazione
MINE	utente	-	Mostra le prenotazioni dell'utente corrente
SEARCH	utente	campo, valore	Cerca tra le prenotazioni dell'utente corrente
ALL	admin	-	Mostra tutte le prenotazioni
STATUS	admin	id, nuovo stato	Modifica lo stato di una prenotazione
CONFLICTS	admin	-	Elenca le prenotazioni potenzialmente in conflitto

Framing delle risposte

TCP fornisce un flusso continuo di byte e non conserva automaticamente i confini dei messaggi. Per questo motivo non si può assumere che una singola chiamata a `recv()` corrisponda sempre a un messaggio applicativo completo.

Per gestire le risposte del server, ho aggiunto un semplice meccanismo di framing: ogni risposta termina con la stringa speciale `END\n`, definita come `MSG_END`. Il client continua a leggere dal socket finché non trova questa sequenza, accodando i dati ricevuti e ricostruendo così anche risposte lunghe che potrebbero arrivare frammentate in più pacchetti TCP.

Per le richieste inviate dal client al server non ho introdotto un terminatore analogo. Questo perché il protocollo implementato è sincrono: il client invia un solo comando alla volta, attende la risposta completa e solo dopo permette l'invio di una nuova richiesta. Inoltre, i comandi previsti hanno dimensione ridotta rispetto al buffer utilizzato.

Strutture dati

Le due entità principali del sistema sono rappresentate dalle strutture `Booking` e `User`, definite in `server.c`.

La struttura `Booking` contiene tutte le informazioni associate a una prenotazione: identificativo, risorsa richiesta, data, orario di inizio e fine, username del richiedente e stato corrente della richiesta.

```
typedef struct {
    int id;
    char resource[50];
    char date[20];
    char time_start[10];
    char time_end[10];
    char requester[50];
    char status[20];
} Booking;
```

Lo stato di una prenotazione può assumere tre valori: `pending` per le richieste in attesa di valutazione, `approved` per le richieste approvate, `refused` per le richieste rifiutate.

La struttura `User` memorizza invece le informazioni degli utenti registrati.

```
typedef struct {  
    int id;  
    char username[50];  
    char password[50];  
} User;
```

Gli utenti e le prenotazioni vengono gestiti tramite due array statici di dimensione massima pari a `MAX_USERS` e `MAX_BOOKINGS`. Così si da mantenere semplice la gestione della memoria.

Per assegnare identificativi numerici univoci ho utilizzato due contatori incrementali, `next_user_id` e `next_booking_id`. All'avvio del server, dopo il caricamento dei dati dai file, i contatori vengono aggiornati in modo da ripartire dal primo identificativo disponibile e non riutilizzare ID già assegnati.

Gestione della concorrenza

Il server crea un thread per ogni client connesso. Di conseguenza, più thread possono tentare nello stesso momento di leggere o modificare gli array di utenti e prenotazioni. Senza un meccanismo di sincronizzazione, potrebbero verificarsi `race condition`, ad esempio durante la registrazione di un nuovo utente, l'inserimento di una prenotazione o la modifica dello stato di una richiesta.

Per evitare questi problemi ho utilizzato un mutex POSIX globale:

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

Le operazioni che accedono a dati condivisi acquisiscono il lock tramite `pthread_mutex_lock()` e lo rilasciano con `pthread_mutex_unlock()`. In questo modo una sola operazione critica alla volta può modificare lo stato interno del server.

Ho usato un unico mutex globale per rendere le cose semplici anche se sarebbe stato più efficiente usare 2 lock separati per utenti e prenotazioni, ma avrei aumentato la complessità senza portare vantaggi significativi.

Gestione dei conflitti

Ho implementato il controllo dei conflitti interamente lato server, in modo che la consistenza dei dati non dipenda dal comportamento dei client.

Due prenotazioni vengono considerate in conflitto quando riguardano la stessa risorsa, nella stessa data, e presentano una sovrapposizione tra gli intervalli temporali richiesti. Per confrontare gli orari, il server converte le stringhe nel formato `hh:mm` in minuti e controlla la sovrapposizione tra intervalli:

```
inizio_a < fine_b && inizio_b < fine_a
```

Il sistema permette che più richieste in attesa si sovrappongano tra loro, perché una richiesta pendente non rappresenta ancora una prenotazione effettivamente confermata. Sarà poi l'amministratore a decidere quali richieste approvare e quali rifiutare.

Dunque il sistema impone che non possono esistere due prenotazioni approvate relative alla stessa risorsa, nella stessa data e con intervalli temporali sovrapposti.

Il controllo per assicurarsi di ciò viene fatto in due momenti:

1. durante l'inserimento di una nuova richiesta, il server rifiuta la prenotazione solo se essa va in conflitto con una prenotazione già approvata.
2. durante l'approvazione di una richiesta pendente, il server controlla nuovamente che non esista un'altra prenotazione approvata in conflitto.

Il controllo lato server è importante, perché tra il momento in cui una richiesta viene inserita e il momento in cui viene approvata il server potrebbe aver approvato altre prenotazioni concorrenti. Eseguo il controllo e il cambio di stato all'interno della stessa sezione critica protetta dal mutex, quindi l'operazione risulta atomica dal punto di vista logico.

Il comando `CONFLICTS` permette all'amministratore di visualizzare con facilità le coppie di prenotazioni che risultano in conflitto, escludendo quelle rifiutate. Questa funzionalità è utile soprattutto per individuare richieste pendenti incompatibili tra loro e decidere manualmente quali approvare.

Persistenza dei dati

Per evitare che utenti e prenotazioni vengano persi alla terminazione del server, ho implementato una semplice persistenza su file di testo.

Salvo i dati in due file:

- `users.txt`, contenente gli utenti registrati.
- `bookings.txt`, contenente le prenotazioni.

Ho usato per i file un formato testuale basato sul separatore `|`, seguendo lo stesso principio adottato nel protocollo. Questa scelta rende i file facilmente leggibili e semplifica le operazioni di caricamento e salvataggio.

All'avvio del server i dati vengono caricati dal file alla memoria. Durante questa fase vengono aggiornati anche i contatori degli ID, così da mantenere la coerenza degli identificativi dopo il riavvio del programma.

Ogni volta che avviene una modifica rilevante, come la registrazione di un utente, l'inserimento di una prenotazione o il cambio di stato di una richiesta, il file corrispondente viene riscritto assicurando la persistenza.

Autenticazione e autorizzazione

Il sistema distingue due tipologie di utenti: utenti standard e amministratore.

Gli utenti standard possono registrarsi tramite il comando `REGISTER` e accedere con il comando `LOGIN`. Dopo l'autenticazione possono inviare nuove richieste di prenotazione tramite il comando `BOOK`, visualizzare le proprie prenotazioni e cercare tra esse tramite il comando `SEARCH`.

L'amministratore accede invece utilizzando credenziali predefinite (`admin / admin`) e, una volta autenticato, può utilizzare i comandi riservati `ALL`, `STATUS` e `CONFLICTS`.

Lo stato di autenticazione è mantenuto localmente nel thread che gestisce il client. In particolare, ogni thread conserva lo username dell'utente autenticato e un flag che indica se la sessione appartiene all'amministratore.

Il controllo delle autorizzazioni viene fatto nel ciclo principale di gestione del client. Prima del login vengono ammessi solo i comandi `REGISTER` e `LOGIN`. Qualsiasi altra operazione produce un errore di accesso non autorizzato. Analogamente, i comandi amministrativi sono autorizzati solo se il flag di amministratore è attivo.

Per rispettare il requisito di riservatezza, ogni utente standard può consultare esclusivamente le proprie prenotazioni. Le funzioni che implementano MINE e SEARCH filtrano i risultati confrontando il campo `requester` della prenotazione con lo `username` dell'utente autenticato.

Solo l'amministratore, tramite il comando ALL, ha visibilità completa sull'archivio delle prenotazioni. In questo modo il sistema impedisce a un utente standard di accedere ai dati relativi ad altri utenti.

Gestione degli errori

Nel progetto ho gestito diverse situazioni di errore comuni, sia lato client sia lato server.

Sul lato server, ogni comando verifica la presenza degli argomenti richiesti e restituisce un messaggio di errore se la sintassi non è corretta. Ho inoltre aggiunto controlli sul formato della data, sul formato degli orari e sulla coerenza dell'intervallo temporale, in modo da impedire richieste con ora di fine precedente o uguale all'ora di inizio.

Ho gestito anche i casi in cui un record non venga trovato, ad esempio durante la ricerca di una prenotazione o durante la modifica dello stato tramite ID. Se l'utente tenta di eseguire un comando senza essere autenticato, oppure tenta di usare comandi riservati all'amministratore, il server restituisce un errore di autorizzazione.

Per quanto riguarda gli errori di rete, sia client sia server controllano il valore di ritorno delle chiamate `send()` e `recv()`. Se la connessione viene chiusa o si verifica un errore, il programma gestisce la situazione chiudendo il socket interessato. Il server ignora inoltre il segnale SIGPIPE, evitando che il processo termini nel caso in cui tenti di scrivere su un socket già chiuso dal client.

Ho considerato anche alcuni limiti pratici dell'implementazione, come il numero massimo di utenti e prenotazioni e la dimensione massima delle risposte. Costruisco le risposte in buffer di dimensione fissa e limito le concatenazioni per evitare overflow. Nel caso di archivi molto grandi, una risposta particolarmente lunga potrebbe essere troncata, ma per questo progetto questo limite è accettabile.

Limiti e possibili estensioni

Il primo limite riguarda l'uso di array statici per utenti e prenotazioni. Questa soluzione è sufficiente per il progetto, ma in un sistema reale sarebbe meglio usare strutture dinamiche o un database.

Un altro limite riguarda la gestione delle password, che nella versione attuale sono salvate in chiaro. Una possibile estensione sarebbe quindi introdurre il salvataggio tramite hash.

Infine, il protocollo potrebbe essere reso più robusto aggiungendo un framing esplicito anche per le richieste del client. Altre estensioni possibili sono l'aggiunta di un'interfaccia grafica, un sistema di log e una gestione più avanzata di date e orari.